

Manual de Programador

Sistema de Tienda de Videojuegos

Davidson Sanchez Gordillo – 20231020183
Andrés Felipe Rondón Lara – 20232020005
Santiago Andrés Benavides Coral – 20232020036

I. Introducción

El presente documento describe el funcionamiento interno del sistema de tienda de videojuegos, desarrollado como una API REST. Permite la gestión de usuarios, carritos de compra, pedidos y procesamiento de pagos.

El objetivo es proporcionar a los desarrolladores una comprensión clara de la arquitectura del sistema, sus componentes principales y la forma en que interactúan entre sí, facilitando el mantenimiento, escalabilidad y futuras mejoras.

El sistema está diseñado siguiendo principios de arquitectura en capas, separando responsabilidades en controladores, servicios, repositorios y entidades.

II. Tecnologías Utilizadas

- **Java** — Lenguaje principal de desarrollo.
- **Spring Boot** — Framework para la API REST.
- **Maven** — Gestión de dependencias y construcción.
- **MySQL / PostgreSQL** — Base de datos relacional.
- **Docker** — Contenedorización del entorno.

III. Estructura del Proyecto

El proyecto sigue una arquitectura en capas, organizada por responsabilidad:

- **config** — Configuraciones generales del sistema (CORS, Swagger).
- **controllers** — Maneja las solicitudes HTTP (endpoints REST).
- **dto** — Define los objetos de transferencia de datos.
- **entities** — Representa las entidades del dominio del sistema.
- **repositories** — Gestiona el acceso a la base de datos.
- **security** — Implementa autenticación y autorización JWT.
- **services** — Contiene la lógica de negocio del sistema.

IV. Modelo del Sistema

En la Figura 1 se presenta el diagrama de clases del sistema, describiendo las entidades principales y sus relaciones.

Las principales entidades del sistema son:

- **Usuario** — Credenciales y rol del usuario en el sistema.

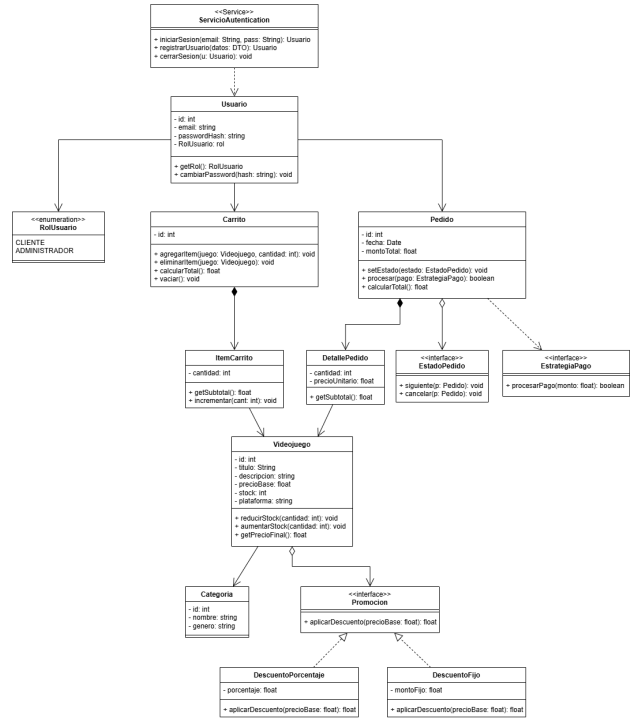


Figura 1. Diagrama de clases del sistema

- **Carrito** — Videojuegos seleccionados antes de una compra.
- **Pedido** — Compra realizada por un usuario.
- **Videojuego** — Producto principal del catálogo.
- **DetallePedido** — Información detallada por producto dentro de un pedido.

V. Flujo del Sistema

El proceso general de compra sigue estos pasos:

1. El usuario inicia sesión en el sistema.
2. Agrega videojuegos al carrito.
3. El sistema calcula el total de la compra.
4. Se genera un pedido a partir del carrito.
5. Se procesa el pago mediante una estrategia de pago.
6. Se actualiza el estado del pedido.

VI. Endpoints Principales

- POST /auth/login — Inicia sesión y retorna token JWT.
- POST /auth/register — Registra un nuevo usuario.
- GET /videojuegos — Obtiene el catálogo con filtros opcionales.
- POST /carrito/agregar — Agrega un videojuego al carrito.
- POST /pedido/crear — Genera un pedido desde el carrito.

VII. Base de Datos

El sistema utiliza una base de datos relacional estructurada mediante los archivos `schema.sql` y `data.sql`. Las tablas principales son: *Usuario*, *Videojuego*, *Pedido* y *DetallePedido*.

VIII. Seguridad

El sistema implementa mecanismos de autenticación y autorización mediante credenciales de usuario y el uso de tokens JWT, lo que permite gestionar sesiones de manera segura y escalable. Las contraseñas se almacenan utilizando un algoritmo de hash seguro (BCrypt), evitando el almacenamiento de credenciales en texto plano.

Adicionalmente, el sistema maneja roles de usuario (ADMIN, CLIENTE) que permiten controlar el acceso a las distintas funcionalidades, garantizando que cada usuario solo pueda acceder a los recursos permitidos según su perfil.

Como parte de las buenas prácticas de desarrollo seguro, el código fue analizado utilizando la herramienta SonarQube, la cual permitió identificar posibles vulnerabilidades, errores y malas prácticas en el código fuente. Tras la ejecución del análisis, se verificó que el sistema cumple con los estándares de calidad y no presenta fallos críticos de seguridad, lo que contribuye a mejorar la confiabilidad del sistema.

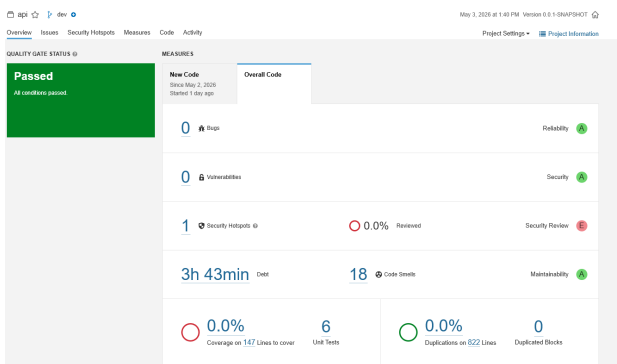


Figura 2. Resultado del análisis de calidad y seguridad del código mediante SonarQube

El análisis realizado con SonarQube (ver Figura 2) evidencia que el sistema no presenta vulnerabilidades críticas ni errores graves, cumpliendo con estándares adecuados de calidad y seguridad del código.

IX. Ejecución del Proyecto

Para ejecutar correctamente el proyecto, se deben seguir los siguientes pasos:

Prerrequisitos

- Tener instalado Java (versión 17 o superior).
- Tener instalado Maven.
- Contar con un gestor de base de datos (por ejemplo, MySQL o PostgreSQL).
- Tener acceso a internet para la descarga de dependencias.

Pasos de ejecución

1. Clonar el repositorio:

```
git clone <URL_DEL_REPOSITORIO>
cd <NOMBRE_DEL_PROYECTO>
```

2. Configurar la base de datos en el archivo `application.properties`, incluyendo:

- URL de conexión
- Usuario y contraseña
- Driver correspondiente

3. Crear la base de datos en el gestor correspondiente (si no existe).

4. Instalar las dependencias del proyecto:

```
mvn clean install
```

5. Ejecutar la aplicación:

```
mvn spring-boot:run
```

Acceso al sistema

Una vez ejecutado, el sistema estará disponible en:

`http://localhost:8080`

Consideraciones

- Verificar que el puerto 8080 no esté en uso.
- En caso de errores, revisar las credenciales de la base de datos.
- Asegurarse de que las variables de entorno estén correctamente configuradas si el proyecto las utiliza.

X. Descripción del Código de Backend

El backend sigue una arquitectura en capas que separa responsabilidades y facilita mantenimiento, escalabilidad y pruebas. La estructura principal se divide en los siguientes componentes.

X-A. Api

Módulo principal del proyecto, organizado en paquetes según su responsabilidad.

X-A1. config

Contiene configuraciones globales del sistema, especialmente para documentación, integración y comunicación con clientes externos. Desacopla estos aspectos del resto de la lógica de negocio.

X-A1a. *SwaggerConfig*

Configura la documentación interactiva de la API mediante OpenAPI/Swagger, permitiendo visualizar y probar endpoints. También define el esquema de autenticación JWT visible desde la interfaz de Swagger UI:

```
@OpenAPIDefinition(  
    info=@Info(  
        title="VideoGames Store API",  
        version="1.0"  
    )  
)  
@SecurityScheme(  
    name="bearerAuth",  
    scheme="bearer",  
    bearerFormat="JWT"  
)
```

Esta configuración aporta:

- Documentación automática de endpoints.
- Pruebas interactivas desde Swagger UI.
- Integración y visibilidad del esquema de autenticación JWT.

X-A1b. *WebConfig*

Configura las políticas CORS (Cross-Origin Resource Sharing), necesarias para permitir la comunicación entre el frontend y el backend. Se habilitan los orígenes usados por frameworks como React, Angular y Vite:

```
registry.addMapping("/api/**")  
    .allowedOrigins(  
        "http://localhost:3000",  
        "http://localhost:4200"  
    )  
    .allowedMethods(  
        "GET", "POST", "PUT", "DELETE"  
    )  
);
```

Permite comunicación segura entre capas, control de orígenes y soporte para peticiones autenticadas con credenciales.

X-A2. controllers

Contiene los controladores REST encargados de exponer los endpoints y actuar como punto de entrada para las solicitudes del cliente. Implementan el patrón MVC en la capa de presentación, delegando la lógica de negocio a los servicios.

X-A2a. *AuthController*

Gestiona los procesos de autenticación: registro, validación de credenciales e inicio de sesión. Delega el procesamiento al servicio AuthService.

```
@PostMapping("/login")  
public ResponseEntity<?> login(  
    @RequestBody LoginRequest req) { ... }  
  
@PostMapping("/registro")  
public ResponseEntity<?> registrar(...) { ... }
```

Sus responsabilidades incluyen:

- Registro de nuevos usuarios.
- Validación de credenciales.
- Inicio de sesión con retorno de token JWT.
- Manejo de errores de acceso.

X-A2b. *PedidoController*

Administra las operaciones relacionadas con compras y pedidos. El usuario autenticado se obtiene directamente desde el contexto de seguridad, garantizando que los pedidos siempre estén asociados a un usuario válido:

```
@PostMapping("/checkout")  
public ResponseEntity<?> realizarCompra(...) {  
    Authentication auth =  
        SecurityContextHolder  
            .getContext()  
            .getAuthentication();  
    ...  
}
```

Incluye endpoints diferenciados por rol:

- Historial de compras para clientes.
- Consulta global de todos los pedidos para administradores.

X-A2c. *VideojuegoController*

Administra las operaciones CRUD del catálogo con soporte para filtros dinámicos. Los filtros disponibles son por título, categoría, plataforma y precio máximo:

```
@GetMapping // listar con filtros  
@PostMapping // crear videojuego  
@PutMapping("/{id}") // actualizar  
@DeleteMapping("/{id}") // eliminar
```

Toda la lógica de filtrado y persistencia se delega a VideojuegoService.

X-A2d. *Consideraciones de diseño*

Los controladores siguen estrictamente la separación de responsabilidades: reciben solicitudes HTTP, validan entradas, delegan la lógica a servicios y retornan respuestas estructuradas. Se integran con Spring MVC, Spring Security, DTOs y arquitectura REST.

X-A3. dto

Los DTO (Data Transfer Objects) transportan información entre cliente, controladores y servicios. Su propósito es desacoplar las entidades del dominio de los datos intercambiados por la API, evitando exponer directamente la estructura interna del modelo.

X-A3a. *LoginRequest*

Encapsula las credenciales necesarias para el inicio de sesión:

```
@Data
public class LoginRequest {
    private String email;
    private String password;
}
```

X-A3b. *RegistroRequest*

Captura los datos requeridos para crear un nuevo usuario:

```
public class RegistroRequest {
    private String nickname;
    private String email;
    private String password;
}
```

X-A3c. *ItemCarritoDTO*

Representa un producto dentro de una solicitud de compra:

```
public class ItemCarritoDTO {
    private Integer idVideojuego;
    private Integer cantidad;
}
```

X-A3d. *PedidoRequest*

Encapsula la solicitud completa de checkout, agrupando múltiples ítems. El correo del usuario **no** se envía en el cuerpo: se extrae del token JWT para prevenir manipulación desde el cliente:

```
public class PedidoRequest {
    private List<ItemCarritoDTO> items;
}
```

X-A3e. *Uso de Lombok*

Todos los DTOs usan la anotación `@Data` de Lombok, que genera automáticamente `getters`, `setters`, `toString()`, `equals()` y `hashCode()`, reduciendo código repetitivo y mejorando mantenibilidad.

X-A4. entities

Representan el modelo del dominio y su mapeo relacional con la base de datos mediante JPA. Cada entidad usa anotaciones como `@Entity`, `@Table`, `@Id` y `@GeneratedValue` para definir persistencia y correspondencia con las tablas.

X-A4a. *Usuario y Rol*

La entidad `Usuario` almacena credenciales y permisos. Se relaciona con `Rol` mediante una asociación *ManyToOne*, permitiendo que múltiples usuarios compartan un rol. Los roles disponibles son `ADMIN` y `CLIENTE`:

```
@ManyToOne
@JoinColumn(name="rol")
private Rol rol;
```

X-A4b. *Videojuego*

Entidad principal del catálogo. Atributos: título, descripción, precio base y stock:

```
@Column(name="precio_base")
private Double precioBase;
```

X-A4c. *Categoria y Plataforma*

Clasifican los videojuegos mediante relaciones *ManyToMany*, usando las tablas pivote `vj_cat` y `vj_plat`. Un videojuego puede pertenecer a múltiples categorías y plataformas:

```
@ManyToMany
@JoinTable(
    name="vj_cat", ...
)
private List<Categoria> categorias;
```

X-A4d. *Pedido*

Representa una compra realizada. Contiene fecha, estado, total, usuario asociado y lista de detalles. Usa `CascadeType.ALL` para persistencia automática de detalles y `FetchType.EAGER` para recuperación inmediata:

```
@OneToMany(
    mappedBy="pedido",
    cascade=ALL
)
private List<DetallePedido> detalles;
```

X-A4e. *DetallePedido*

Representa cada línea de producto dentro de un pedido: videojuego comprado, cantidad y precio unitario. Para evitar ciclos en la serialización JSON se usa `@JsonIgnore` en la referencia al pedido padre:

```
@ManyToOne
@JoinColumn(name="id_vj")
private Videojuego videojuego;
```

X-A4f. *Resumen de relaciones*

- *ManyToOne*: Usuario → Rol
- *OneToMany*: Pedido → DetallePedido
- *ManyToMany*: Videojuego ↔ Categoria
- *ManyToMany*: Videojuego ↔ Plataforma

X-A5. repositories

Implementan la capa de acceso a datos mediante Spring Data JPA. Todas las interfaces extienden `JpaRepository<T, ID>`, obteniendo operaciones CRUD automáticas (guardar, buscar, actualizar, eliminar) sin escribir SQL manual.

X-A5a. *UsuarioRepository*

Provee búsqueda por correo electrónico, utilizada principalmente en autenticación:

```
Optional<Usuario> findByEmail(String email);
```

X-A5b. *RolRepository*

Recupera roles por nombre para asignación durante el registro:

```
Optional<Rol> findByRol(String rol);
```

X-A5c. *PedidoRepository*

Permite consultar el historial de compras navegando propiedades relacionadas, sin necesidad de joins explícitos:

```
List<Pedido> findByUsuarioEmail(String email);
```

X-A5d. *VideojuegoRepository*

Implementa filtros dinámicos derivados por nombre de método. Spring genera el SQL correspondiente automáticamente:

```
findByTituloContainingIgnoreCase(String titulo)
findByPrecioBaseLessThanEqual(Double precio)
findByCategoriasNombreIgnoreCase(String nombre)
findByPlataformasNombreIgnoreCase(String nombre)
```

Este mecanismo mejora legibilidad, productividad y mantenibilidad sin sacrificar flexibilidad de consulta. Los servicios consumen repositorios sin acoplarse a detalles de base de datos.

X-A6. security

Implementa la arquitectura de seguridad basada en JWT y autorización por roles con Spring Security. Controla acceso, protección de rutas y validación de usuarios autenticados. Se compone de tres elementos: `JwtFilter`, `JwtUtil` y `SecurityConfig`.

X-A6a. *JwtFilter*

Extiende `OncePerRequestFilter`, garantizando ejecución una sola vez por solicitud. Intercepta cada petición HTTP para leer el token del encabezado `Authorization`, validarlo y registrar al usuario en el contexto de seguridad:

```
String authHeader =
    request.getHeader("Authorization");

if(authHeader.startsWith("Bearer ")) {
    String token = authHeader.substring(7);
}
```

```
UsernamePasswordAuthenticationToken auth =
    new UsernamePasswordAuthenticationToken(
        email, null, authorities);
SecurityContextHolder.getContext().
    setAuthentication(auth);
```

Esto permite que cada endpoint conozca el usuario autenticado sin necesidad de sesiones del lado del servidor.

X-A6b. *JwtUtil*

Centraliza la generación y validación de tokens. El token almacena email, rol, fecha de emisión y fecha de expiración. La duración configurada es de dos horas (7 200 000 ms):

```
Jwts.builder()
    .setSubject(email)
    .claim("rol", rol)
    .signWith(SECRET_KEY)
    .compact();
```

```
public boolean validarToken(...) { ... }
```

La validación evita acceso mediante tokens inválidos o expirados.

X-A6c. *SecurityConfig*

Define la política de seguridad completa del sistema. Sus responsabilidades principales son:

- Configurar rutas públicas y privadas.
- Definir permisos diferenciados por rol.
- Registrar el filtro JWT antes del filtro estándar de Spring.
- Configurar `BCryptPasswordEncoder` para contraseñas.

Ejemplo de restricción por rol:

```
.requestMatchers(
    HttpMethod.POST, "/api/videojuegos/**"
).hasAuthority("ADMIN")
```

El sistema opera en modo *stateless*, sin sesiones del servidor:

```
SessionCreationPolicy.STATELESS
```

```
addFilterBefore(
    jwtFilter,
    UsernamePasswordAuthenticationFilter.class
)
```

X-A6d. *Control de acceso por roles (RBAC)*

- Login y registro: **público**.
- Catálogo de videojuegos: **público (solo lectura)**.
- Gestión del catálogo: solo **ADMIN**.
- Pedidos y checkout: solo usuarios **autenticados**.
- Documentación Swagger: **pública**.

X-A6e. *Flujo de autenticación*

1. Usuario inicia sesión con credenciales.
2. Se genera y retorna un token JWT.

3. El cliente envía el token en el encabezado de cada solicitud.
4. `JwtFilter` intercepta y valida el token.
5. El `SecurityContext` registra al usuario autenticado.
6. Spring Security evalúa permisos y autoriza (o rechaza) el acceso.

X-A7. services

Implementan la capa de lógica de negocio. Actúan como intermediario entre controladores y repositorios, procesando reglas del dominio y coordinando entidades. Siguen el patrón *Service Layer*.

X-A7a. AuthService

Gestiona registro de usuarios, autenticación y generación de tokens.

Todo usuario registrado recibe automáticamente el rol `CLIENTE`:

```
Rol rolCliente =
    rolRepository.findByRol("CLIENTE")
        .orElseThrow(...);
```

Las contraseñas se almacenan y verifican con cifrado:

```
passwordEncoder.encode(req.getPassword())
passwordEncoder.matches(rawPassword, hash)
```

Si las credenciales son válidas, se genera y retorna el JWT:

```
jwtUtil.generarToken(email, rol)
```

X-A7b. PedidoService

Implementa la lógica completa de compra. Está marcado como `@Transactional` para garantizar integridad: si una operación falla, la transacción puede revertirse automáticamente.

Secuencia de ejecución:

1. Validar que el carrito no esté vacío.
2. Verificar stock disponible para cada ítem.
3. Descontar inventario.
4. Construir el pedido y sus detalles.
5. Calcular el total.
6. Persistir el pedido (los detalles se guardan en cascada).

```
if(vj.getStock() < item.getCantidad())
    throw new RuntimeException(...);
```

```
vj.setStock(
    vj.getStock() - item.getCantidad());
```

```
total += precio * cantidad;
pedidoRepository.save(pedido);
```

El guardado usa `CascadeType.ALL`, por lo que los `DetallePedido` se persisten automáticamente con el pedido padre.

También soporta consulta de historial por cliente y listado global para administradores.

X-A7c. VideojuegoService

Administra el catálogo: CRUD y filtrado dinámico. Los filtros se aplican en memoria usando Streams de Java sobre la lista recuperada del repositorio:

```
public List<Videojuego>
    obtenerFiltrados(String titulo,
        String categoria, String plataforma,
        Double precioMax) {

    lista.stream()
        .filter(...)
        .collect(Collectors.toList());
}
```

Esto permite combinaciones de filtros flexibles sin múltiples consultas especializadas. Ejemplo de actualización:

```
vj.setTitulo(...);
vj.setPrecioBase(...);
repository.save(vj);
```

X-A7d. Flujo general de negocio

1. El controlador recibe la solicitud.
2. El servicio valida reglas de negocio.
3. Consulta o actualiza los repositorios necesarios.
4. Persiste los cambios.
5. Retorna el resultado al controlador.

X-B. Resources

El directorio `resources` contiene los archivos de configuración, inicialización y estructura de la base de datos. Permiten configurar el entorno de ejecución, crear el esquema relacional e insertar datos iniciales.

X-B1. application.properties

Centraliza la configuración principal del backend. Incluye parámetros de conexión a la base de datos, configuración JPA/Hibernate e inicialización automática de scripts SQL:

```
spring.datasource.url=
    jdbc:postgresql://localhost:5433/
    videogames_store
spring.datasource.username=santiago_dev
spring.datasource.password=password123
```

```
spring.jpa.hibernate.ddl-auto=none
spring.sql.init.mode=always
spring.jpa.show-sql=true
```

`ddl-auto=none` deshabilita la autocreación de tablas por Hibernate para evitar interferencia con el esquema definido manualmente. `init.mode=always` ejecuta `schema.sql` y `data.sql` automáticamente en cada arranque. `show-sql=true` permite visualizar las consultas generadas por JPA durante la ejecución.

X-B2. data.sql

Contiene los datos semilla para poblar la base de datos en el arranque del sistema. Incluye:

- **Roles:** ADMIN y CLIENTE, necesarios para autenticación y autorización.
- **Categorías:** Acción, Aventura, RPG.
- **Plataformas:** PC, PS5, Xbox Series.
- **Videojuego inicial:** *Cyber Night 2077* como producto de prueba.
- **Tablas pivot:** asociaciones iniciales en `vj_cat` y `vj_plat`.
- **Secuencias:** ajuste de autoincrementales para evitar conflictos con futuros registros (`SELECT setval(...)`).

Este archivo garantiza consistencia del entorno para desarrollo y pruebas reproducibles.

X-B3. schema.sql

Define manualmente la estructura completa de la base de datos. Se ejecuta antes de `data.sql` y comienza con la eliminación del esquema existente:

```
DROP TABLE IF EXISTS ... CASCADE;
```

Tablas definidas: *roles*, *usuarios*, *videojuegos*, *pedidos*, *detalle_pedido*, *categorias*, *plataformas*, *vj_cat* y *vj_plat*.

Se implementan restricciones de integridad de dominio:

```
CHECK(stock >= 0)
CHECK(cantidad > 0)
estado IN ('Completado', 'Cancelado')
```

Relaciones mediante llaves foráneas:

```
FOREIGN KEY (rol) REFERENCES roles(id_rol)
```

Las tablas pivot usan claves compuestas para modelar relaciones ManyToMany:

```
PRIMARY KEY(id_vj, id_cat)
```

Se emplean políticas ON DELETE CASCADE y ON DELETE RESTRICT para controlar la integridad referencial según la criticidad de cada relación.

El esquema implementa normalización relacional, restricciones de dominio y sirve como base estructural del sistema.

X-C. Test

La carpeta `test` contiene las pruebas automatizadas orientadas a verificar funcionalidad, validaciones de negocio y rendimiento de los servicios principales. Se utiliza JUnit 5 con Spring Boot Test.

X-C1. ApiApplicationTests

Clase principal de pruebas, declarada con las siguientes anotaciones:

```
@SpringBootTest
```

```
@Transactional
```

`@SpringBootTest` carga el contexto completo de la aplicación. `@Transactional` revierte los cambios en BD tras cada prueba, garantizando aislamiento y reproducibilidad.

X-C1a. Pruebas funcionales de autenticación

- **Registro exitoso:** verifica que un usuario válido puede registrarse sin errores.

```
assertDoesNotThrow(
    () -> authService.registrar(req));
```

- **Correo duplicado:** valida que el sistema impide registrar el mismo correo dos veces.

```
assertThrows(
    RuntimeException.class, ...);
```

- **Login inválido:** credenciales incorrectas deben lanzar una excepción controlada, probando robustez en autenticación.

X-C1b. Pruebas del catálogo

Se invoca el método de filtrado sin parámetros y se verifica que la respuesta no sea nula:

```
List<Videojuego> catalogo =
    videojuegoService.obtenerFiltrados(
        null, null, null, null);
assertNotNull(catalogo);
```

X-C1c. Pruebas de rendimiento

Se verifica que los servicios respondan dentro de tiempos aceptables, usando `assertTimeout`:

```
assertTimeout(
    Duration.ofMillis(500), () -> {
        videojuegoService.obtenerFiltrados(
            null, "RPG", null, null);
    });
```

Permite detectar regresiones de rendimiento de forma temprana.

X-C1d. Cobertura general

Las pruebas cubren: registro correcto, duplicidad de usuarios, login inválido, consulta del catálogo con y sin filtros, y desempeño básico de servicios. La estrategia integra pruebas funcionales, de validación, negativas, de integración y de rendimiento básico.

XI. Descripción del Código Frontend

El frontend del sistema fue desarrollado como una aplicación web modular orientada a la interacción entre usuarios y la API REST del backend.

La aplicación implementa separación de responsabilidades mediante componentes reutilizables, páginas independientes y estilos organizados.

La estructura principal se encuentra dentro del directorio:

`src`

el cual contiene los módulos fundamentales de la interfaz gráfica.

XI-A. `src`

La carpeta `src` centraliza el código fuente del frontend.

Su organización permite separar lógica visual, navegación, estilos y componentes reutilizables.

Se compone de los siguientes directorios principales:

- `components`
- `content`
- `layouts`
- `lib`
- `pages`
- `styles`

XI-A1. `components`

La carpeta `components` contiene los componentes reutilizables de la interfaz gráfica.

Estos componentes encapsulan lógica visual y funcionalidad específica para construir páginas dinámicas mediante una arquitectura modular.

Entre sus ventajas se encuentran:

- reutilización de código
- separación de responsabilidades
- mantenimiento simplificado
- consistencia visual

XI-A1a. `GameCard`

El componente `GameCard` representa cada videojuego dentro del catálogo.

Muestra:

- imagen
- título
- descripción
- precio
- descuentos
- disponibilidad

El componente recibe un objeto de tipo:

`Videojuego`

mediante propiedades.

Ejemplo:

```
interface Props {  
  juego: Videojuego;  
}
```

El sistema calcula automáticamente descuentos.

Ejemplo:

```
const precioRebajado =
```

```
juego.precio *  
(1 - juego.descuento / 100);
```

También se implementa una imagen por defecto en caso de error:

```
const imagenFinal =  
juego.imagenUrl ||  
"/placeholder.jpg";
```

El botón de compra se deshabilita cuando no existe stock.

Ejemplo:

```
disabled={juego.stock <= 0}
```

Visualmente se utilizan clases de Tailwind CSS para diseño responsive y animaciones.

XI-A1b. `HeroSection`

El componente `HeroSection` implementa la sección principal de bienvenida del sistema.

Recibe propiedades dinámicas:

```
interface Props {  
  title: string;  
  subtitle: string;  
  ctaLabel: string;  
  ctaHref: string;  
}
```

Permite mostrar:

- título principal
- subtítulo
- barra de búsqueda
- botón de acción

Ejemplo del botón CTA:

```
<a href={ctaHref}>
```

El diseño utiliza gradientes, efectos blur y distribución responsive mediante Tailwind CSS.

XI-A1c. `Navbar`

El componente `Navbar` implementa la barra de navegación principal del sistema.

Incluye acceso a:

- inicio
- catálogo
- checkout
- historial
- administración

Los enlaces se almacenan dinámicamente.

Ejemplo:

```
const links = [  
  { href: "/", label: "Inicio" }  
];
```

La barra detecta automáticamente la ruta activa.

Ejemplo:

```
currentPath === link.href
```

También administra autenticación visual usando JWT.

Ejemplo:

```
const token = getToken();
```

Si el usuario ha iniciado sesión, se muestra un menú desplegable con su información y rol.

El cierre de sesión elimina el token almacenado localmente.

Ejemplo:

```
clearToken();
```

XI-A1d. Tecnologías Utilizadas

Los componentes utilizan:

- Astro
- TypeScript
- Tailwind CSS
- JavaScript del cliente

Esto permite construir interfaces modernas, reutilizables y dinámicas.

XI-A2. layouts

La carpeta `layouts` contiene las estructuras base compartidas entre las diferentes páginas del sistema.

Su función principal es mantener una apariencia uniforme y reutilizar elementos globales como:

- metadatos
- estilos globales
- estructura HTML
- footer
- configuración SEO

XI-A2a. Layout Principal

El layout principal importa los estilos globales del sistema.

Ejemplo:

```
import "../styles/global.css";
```

Esto garantiza consistencia visual en toda la aplicación.

El componente recibe propiedades dinámicas para personalizar cada página.

Ejemplo:

```
interface Props {  
  title?: string;  
  description?: string;  
}
```

Si no se proporcionan valores, se utilizan títulos y descripciones por defecto.

XI-A2b. Metadatos y SEO

El layout implementa configuración SEO básica mediante etiquetas meta.

Ejemplo:

```
<meta  
  name="description"  
  content={description}  
>
```

También se generan URLs canónicas dinámicamente.

Ejemplo:

```
const canonical = new URL(  
  Astro.url.pathname,  
  Astro.site ??  
  Astro.url.origin  
).toString();
```

Esto mejora indexación y compatibilidad web.

XI-A2c. Estructura General

El layout define la estructura HTML base de la aplicación.

Ejemplo:

```
<html lang="es">
```

El contenido dinámico de cada página se renderiza mediante:

```
<slot />
```

Esto permite reutilizar el mismo diseño en múltiples vistas.

XI-A2d. Footer Global

El layout incluye un footer compartido para toda la aplicación.

Este contiene:

- información de la tienda
- categorías
- soporte
- contacto

El diseño utiliza grids responsivos mediante Tailwind CSS.

Ejemplo:

```
md:grid-cols-4
```

XI-A2e. Tecnologías Utilizadas

El sistema de layouts utiliza:

- Astro
- HTML semántico
- Tailwind CSS
- metadatos SEO
- componentes reutilizables

Esto permite mantener una estructura consistente y escalable en el frontend.

XI-A3. lib

La carpeta `lib` contiene funciones auxiliares, manejo de datos y definiciones reutilizables utilizadas por el frontend.

Aquí se centraliza:

- comunicación con la API
- manejo de JWT
- almacenamiento local
- tipado TypeScript

XI-A3a. *api.ts*

El archivo `api.ts` centraliza las peticiones HTTP hacia el backend.

Se define una URL base configurable:

```
const API_BASE_URL =
import.meta.env.PUBLIC_API_BASE_URL
?? "http://localhost:8080";
```

La función genérica:

```
request<T>()
```

maneja solicitudes `fetch`, parseo JSON y control de errores.

Ejemplo:

```
throw new ApiError(
  errorMessage,
  response.status
);
```

Los endpoints se organizan por módulos:

- `auth`
- `videojuegos`
- `pedidos`
- `categorias`
- `plataformas`

Ejemplo:

```
api.auth.login(...)
```

XI-A3b. *client-store.ts*

El archivo `client-store.ts` administra el almacenamiento local del navegador.

Se utilizan claves para guardar:

- `token JWT`
- `carrito de compras`

Ejemplo:

```
const TOKEN_KEY = "vgs_token";
```

También se implementa lectura del JWT para extraer información del usuario.

Ejemplo:

```
token.split(".")[1];
```

El carrito se administra mediante funciones como:

- `addToCart()`
- `removeFromCart()`
- `clearCart()`

XI-A3c. *types.ts*

El archivo `types.ts` define las interfaces TypeScript del sistema.

Entre ellas:

- `Videojuego`
- `Pedido`
- `Categoria`
- `Plataforma`
- `LoginRequest`
- `RegistroRequest`

Ejemplo:

```
export interface Videojuego {
  id: number;
  titulo: string;
}
```

Esto mejora organización, tipado y mantenimiento del código.

XI-A3d. *Tecnologías Utilizadas*

La carpeta utiliza:

- TypeScript
- Fetch API
- LocalStorage
- JWT

XI-A4. pages

La carpeta `pages` contiene las páginas principales del sistema.

XI-A4a. *admin.astro*

La página `admin.astro` implementa el panel de administración de *TechZone Gaming*, permitiendo gestionar videojuegos mediante operaciones CRUD.

La vista utiliza los componentes `Layout` y `Navbar`, además de validar el acceso mediante tokens JWT para verificar que el usuario posea el rol ADMIN.

El panel muestra estadísticas generales del inventario utilizando la función:

```
updateStats()
```

Esta función calcula el total de productos, promociones activas y valor total del inventario.

La carga de categorías y plataformas se realiza dinámicamente mediante:

```
loadMetadata()
```

Los videojuegos se representan en una tabla generada con:

```
renderTable()
```

Cada fila incluye opciones para editar o eliminar productos del sistema.

El formulario CRUD permite registrar o actualizar videojuegos ingresando datos como título, precio, stock, descuento, categorías y plataformas.

Finalmente, el evento `submit` decide automáticamente si se debe crear o actualizar un videojuego y posteriormente recarga el inventario para reflejar los cambios realizados.

XI-A4b. catalogo.astro

La página `catalogo.astro` permite visualizar el catálogo principal de videojuegos de *TechZone Gaming*. Incluye filtros dinámicos por título, categoría, plataforma y precio máximo mediante un formulario HTML.

La página utiliza los componentes `Layout` y `Navbar` para mantener la estructura general del sitio. Los filtros se almacenan usando `URLSearchParams`, permitiendo conservar el estado de búsqueda en la URL.

La función principal encargada de cargar los videojuegos es:

```
loadCatalog()
```

Esta función consulta la API aplicando los filtros seleccionados por el usuario.

Los videojuegos se muestran dinámicamente mediante tarjetas generadas por la función:

```
renderCards(juegos)
```

Cada tarjeta presenta información como imagen, precio, descuento, categorías, plataformas y disponibilidad de stock.

Además, cada producto incluye un botón para agregar videojuegos al carrito utilizando:

```
addToCart(gameId)
```

Finalmente, el evento `submit` del formulario actualiza los filtros sin recargar completamente la página, mejorando la experiencia del usuario.

XI-A4c. checkout.astro

La página `checkout.astro` gestiona el proceso final de compra del sistema *TechZone Gaming*.

La vista utiliza los componentes `Layout` y `Navbar`, mostrando el contenido actual del carrito y el total estimado de la compra.

Los productos almacenados en el carrito se cargan mediante:

```
renderCart()
```

Esta función obtiene los videojuegos desde la API, calcula descuentos, subtotales y el total general de la compra.

Cada producto incluye un botón para eliminar elementos del carrito utilizando:

```
removeFromCart(id)
```

El botón `Confirmar compra` ejecuta el proceso de `checkout` mediante:

```
api.pedidos.checkout(token, { items })
```

Antes de procesar la compra, el sistema verifica que exista un usuario autenticado y que el carrito contenga productos.

Además, el botón `Vaciar carrito` utiliza:

```
clearCart()
```

para eliminar todos los productos almacenados localmente.

Finalmente, la página muestra mensajes dinámicos indicando errores, confirmaciones de compra o cambios realizados en el carrito.

XI-A4d. history.astro

La página `history.astro` permite consultar el historial de pedidos realizados por los usuarios autenticados mediante JWT.

La vista utiliza los componentes `Layout` y `Navbar`, manteniendo la estructura general de la aplicación. Dependiendo del rol del usuario, la página consume diferentes endpoints:

```
/api/pedidos/historial  
/api/pedidos/admin/todos
```

La función `loadHistory()` verifica el token y determina si el usuario es cliente o administrador. Si el usuario tiene rol ADMIN, la interfaz cambia dinámicamente para mostrar todos los pedidos registrados en el sistema.

Cada pedido se renderiza dinámicamente mostrando información como ID, estado, total de compra, fecha y videojuegos adquiridos. En el caso del administrador, también se muestran datos del cliente asociados al pedido.

La interfaz maneja mensajes de error y validación para usuarios no autenticados o cuando ocurre un problema al cargar la información desde la API.

XI-A4e. index.astro

La página `index.astro` corresponde al *landing page* principal de *TechZone Gaming*. Su función es presentar promociones, categorías destacadas y acceso rápido al catálogo de videojuegos.

La vista utiliza los componentes `Layout`, `Navbar`, `HeroSection` y `GameCard` para construir la interfaz principal del sistema.

El contenido visual principal se obtiene desde colecciones de Astro Content mediante:

```
getCollection("home")  
getCollection("marketing")
```

Además, la página consume la API del backend para obtener videojuegos destacados:

```
const catalogo = await api.videojuegos.listar();
```

Los primeros seis videojuegos del catálogo se muestran utilizando componentes `GameCard`, permitiendo visualizar promociones y accesos rápidos hacia el catálogo completo.

Finalmente, la página incorpora manejo de errores para mostrar mensajes cuando ocurre un problema al cargar los datos desde el backend.

XI-A4f. login.astro

La página `login.astro` permite a los usuarios autenticarse dentro del sistema mediante correo electrónico y contraseña.

La interfaz utiliza los componentes `Layout` y `Navbar`, además de un formulario HTML para capturar las credenciales del usuario.

El evento `submit` del formulario envía los datos al backend utilizando:

```
api.auth.login({ email, password });
```

Si la autenticación es exitosa, el token JWT se almacena localmente mediante:

```
setToken(token);
```

Finalmente, la página muestra mensajes dinámicos indicando si el inicio de sesión fue exitoso o si ocurrió algún error durante el proceso de autenticación.

XI-A4g. register.astro

La página `register.astro` permite registrar nuevos usuarios dentro de la plataforma *TechZone Gaming*.

La interfaz utiliza los componentes `Layout` y `Navbar`, además de un formulario para capturar el nickname, correo electrónico y contraseña del usuario.

El evento `submit` procesa los datos y los envía al backend mediante:

```
api.auth.registro({ nickname, email, password });
```

Si el registro es exitoso, se muestra un mensaje de confirmación y el formulario se reinicia automáticamente utilizando:

```
form.reset();
```

Finalmente, la página implementa manejo de errores para informar problemas durante el proceso de registro del usuario.

XI-A5. styles

La carpeta `styles` contiene los estilos globales utilizados por toda la aplicación frontend.

XI-A5a. global.css

El archivo `global.css` importa `TailwindCSS` y define configuraciones generales de apariencia.

Ejemplo:

```
@import "tailwindcss";
```

También se establecen estilos base para el documento HTML y el cuerpo de la página:

```
html,
body {
  margin: 0;
  min-height: 100%;
}
```

Además, se configura la fuente principal y el color de fondo general del sistema.

Este archivo permite mantener una apariencia uniforme en todas las páginas del proyecto.

XI-A5b. Tecnologías Utilizadas

La carpeta utiliza:

- CSS
- TailwindCSS

XII. Conclusiones

Durante el desarrollo del proyecto *TechZone Gaming* se logró construir una aplicación web completa orientada a la gestión y venta de videojuegos, integrando tecnologías modernas tanto en el frontend como en el backend.

El sistema fue desarrollado utilizando `Astro` y `TypeScript` para la interfaz de usuario, mientras que `Spring Boot` permitió implementar una API REST segura y organizada para la gestión de usuarios, videojuegos, pedidos y autenticación mediante JWT.

La arquitectura modular utilizada facilitó la separación de responsabilidades entre componentes, páginas, servicios y utilidades, permitiendo mantener un código más escalable, reutilizable y fácil de mantener. Además, el uso de componentes reutilizables mejoró la consistencia visual y redujo la duplicación de código dentro de la aplicación.

Se implementaron funcionalidades importantes como autenticación de usuarios, control de roles, carrito de compras, sistema de checkout, historial de pedidos y panel administrativo con operaciones CRUD completas para la gestión del inventario.

También se incorporó manejo de errores, validaciones y comunicación dinámica con el backend, garantizando una experiencia de usuario más estable y segura.

Finalmente, el desarrollo de este proyecto permitió aplicar conocimientos relacionados con desarrollo full stack, consumo de APIs REST, manejo de estado en el cliente, diseño responsivo y organización profesional de proyectos web modernos.